

Resumen de Arboles Parcialmente Ordenados - Heaps

Introduccion

Los arboles parcialmente ordenados, conocidos como *Heaps*, son estructuras de datos basadas en arboles binarios completos que cumplen una propiedad de orden. En un **Max-Heap**, la raiz contiene el mayor valor de todos los nodos; en un **Min-Heap**, la raiz contiene el menor valor. Esta propiedad hace que los heaps sean ideales para implementar algoritmos de ordenamiento como *HeapSort* y para la gestion eficiente de colas de prioridad.

Conceptos Fundamentales

Arbol Binario Completo

Un arbol binario completo es aquel donde todos los niveles estan completamente llenos, excepto posiblemente el ultimo, que se llena de izquierda a derecha. Esta caracteristica es crucial para los heaps, ya que no admiten *huecos* en su estructura.

Propiedad de Orden

En un heap:

- En un **Max-Heap**, el valor de cada nodo es mayor o igual que los valores de sus hijos.
- En un **Min-Heap**, el valor de cada nodo es menor o igual que los valores de sus hijos.

Representacion en Vector

Los heaps se pueden representar de forma eficiente usando un vector. Dado un nodo en la posicion i :

- Su hijo izquierdo esta en $2i + 1$.
- Su hijo derecho esta en $2i + 2$.
- Su padre esta en $\lfloor (i - 1)/2 \rfloor$.

Operaciones Basicas en Heaps

Encolar

- Se inserta el nuevo elemento al final del vector.
- Se compara con su padre y se intercambia si es necesario.
- Este proceso continua hasta que la propiedad de orden se restablezca.

Desencolar

- Se extrae la raiz del heap (el mayor o menor elemento).
- El ultimo elemento reemplaza la raiz.
- Se ajusta el arbol hacia abajo para restablecer la propiedad de orden.

Ajustar

Ajustar es el proceso de restablecer la propiedad de orden desde un nodo especifico hacia abajo. Esto se realiza comparando el nodo con sus hijos y realizando intercambios si es necesario.

HeapSort

HeapSort es un algoritmo de ordenamiento basado en heaps:

1. Se construye un Max-Heap a partir de los elementos del vector.
2. Se intercambia la raiz con el ultimo elemento y se reduce el tamaño del heap.
3. Se ajusta el heap para restablecer la propiedad de orden.
4. Estos pasos se repiten hasta que el vector este ordenado.

Implementacion Basica de un TDA Heap

Listing 1: Clase Basica de Heap

```
1 public class Heap<E extends Comparable<E>> {
2     private E[] arreglo;
3     private int tamaño;
4
5     public Heap(int capacidad) {
6         arreglo = (E[]) new Comparable[capacidad];
7         tamaño = 0;
8     }
9
10    public void encolar(E elemento) {
11        if (tamaño >= arreglo.length) {
12            throw new IllegalStateException("Heap lleno");
```

```

13     }
14     arreglo[tamano] = elemento;
15     subir(tamano);
16     tamano++;
17 }
18
19 public E desencolar() {
20     if (tamano == 0) {
21         throw new IllegalStateException("Heap vacio");
22     }
23     E raiz = arreglo[0];
24     arreglo[0] = arreglo[--tamano];
25     ajustar(0);
26     return raiz;
27 }
28
29 private void subir(int indice) {
30     while (indice > 0) {
31         int padre = (indice - 1) / 2;
32         if (arreglo[indice].compareTo(arreglo[padre]) <= 0) {
33             break;
34         }
35         intercambiar(indice, padre);
36         indice = padre;
37     }
38 }
39
40 private void ajustar(int indice) {
41     while (indice < tamano / 2) {
42         int izquierdo = 2 * indice + 1;
43         int derecho = 2 * indice + 2;
44         int mayor = izquierdo;
45
46         if (derecho < tamano && arreglo[derecho].compareTo(arreglo[izquierdo])
47             > 0) {
48             mayor = derecho;
49         }
50
51         if (arreglo[indice].compareTo(arreglo[mayor]) >= 0) {
52             break;
53         }
54
55         intercambiar(indice, mayor);
56         indice = mayor;
57     }
58 }
59
60 private void intercambiar(int i, int j) {
61     E temp = arreglo[i];
62     arreglo[i] = arreglo[j];
63     arreglo[j] = temp;
64 }
65
66 public void imprimirHeap() {
67     for (int i = 0; i < tamano; i++) {
68         System.out.print(arreglo[i] + " ");
69     }
70     System.out.println();

```

```
70     }  
71 }
```

Ejemplo de Uso del Heap

Listing 2: Ejemplo de Uso

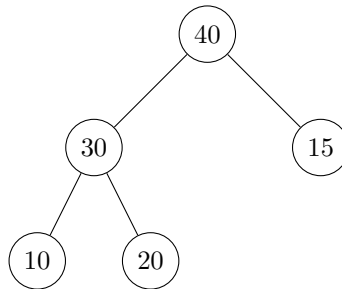
```
1 public class Main {  
2     public static void main(String[] args) {  
3         Heap<Integer> heap = new Heap<>(10);  
4  
5         System.out.println("Encolando elementos:");  
6         heap.encolar(15);  
7         heap.encolar(10);  
8         heap.encolar(20);  
9         heap.encolar(5);  
10  
11        heap.imprimirHeap();  
12  
13        System.out.println("Desencolando elementos:");  
14        System.out.println("Elemento extraido: " + heap.desencolar());  
15        heap.imprimirHeap();  
16  
17        System.out.println("Elemento extraido: " + heap.desencolar());  
18        heap.imprimirHeap();  
19    }  
20 }
```

Ejercicios Faciles

Ejercicio 1: Construcción de un Max-Heap

Dado el conjunto de elementos {10, 20, 15, 30, 40}, construya un Max-Heap insertando los elementos en el orden dado.

Solucion: Inicialmente, el heap esta vacío. Insertamos los elementos uno por uno y ajustamos la propiedad de Max-Heap en cada paso.



Código en Java:

```

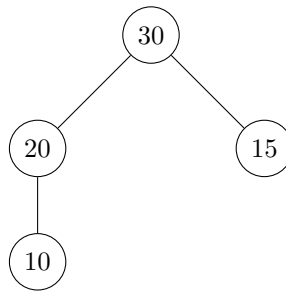
1 public class Main {
2     public static void main(String[] args) {
3         Heap<Integer> heap = new Heap<>(10);
4
5         System.out.println("Insertando elementos:");
6         heap.encolar(10);
7         heap.encolar(20);
8         heap.encolar(15);
9         heap.encolar(30);
10        heap.encolar(40);
11
12        System.out.println("Heap resultante:");
13        heap.imprimirHeap();
14    }
15 }

```

Ejercicio 2: Extraccion de la Raiz de un Max-Heap

Dado el Max-Heap {40, 30, 15, 10, 20}, extraiga la raíz y ajuste el heap para restaurar la propiedad de Max-Heap.

Solucion: La raíz (40) se elimina y el ultimo elemento (20) se mueve a la raíz. Luego, se ajusta el heap.



Codigo en Java:

```

1 public class Main {
2     public static void main(String[] args) {
3         Heap<Integer> heap = new Heap<>(10);
4
5         heap.encolar(40);
6         heap.encolar(30);
7         heap.encolar(15);
8         heap.encolar(10);
9         heap.encolar(20);
10
11        System.out.println("Heap antes de extraer la raiz:");
12        heap.imprimirHeap();
13
14        System.out.println("Elemento extraido: " + heap.desencolar());
15
16        System.out.println("Heap despues de extraer la raiz:");
17        heap.imprimirHeap();
18    }
19 }

```

Ejercicio 3: Verificación de Propiedad de Max-Heap

Determine si el árbol representado por {50, 30, 20, 10, 15, 5} cumple con la propiedad de Max-Heap.

Solucion: El árbol cumple con la propiedad de Max-Heap porque cada nodo es mayor o igual que sus hijos.

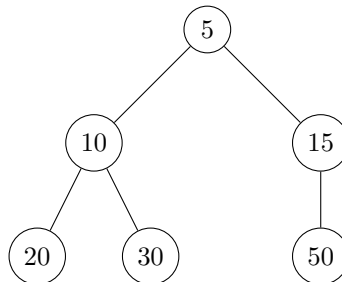
Código en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         Heap<Integer> heap = new Heap<>(10);
4
5         heap.encolar(50);
6         heap.encolar(30);
7         heap.encolar(20);
8         heap.encolar(10);
9         heap.encolar(15);
10        heap.encolar(5);
11
12        System.out.println("Heap construido:");
13        heap.imprimirHeap();
14        System.out.println("El heap cumple con la propiedad de Max-Heap.");
15    }
16 }
```

Ejercicio 4: Construcción de un Min-Heap

Dado el conjunto {50, 30, 20, 10, 15, 5}, construya un Min-Heap.

Solucion: El Min-Heap resultante es:



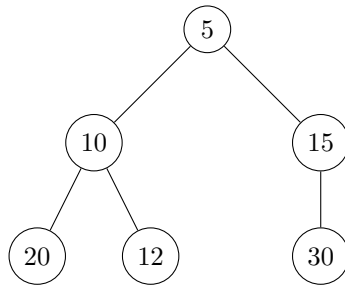
Código en Java:

```
1 public class MinHeap<E extends Comparable<E>> extends Heap<E> {
2     public MinHeap(int capacidad) {
3         super(capacidad);
4     }
5
6     @Override
7     protected boolean comparar(E hijo, E padre) {
8         return hijo.compareTo(padre) < 0;
9     }
10 }
```

Ejercicio 5: Inserción en un Min-Heap

Inserte el elemento 12 en el Min-Heap {5, 10, 20, 30, 15}.

Solucion: El Min-Heap después de la inserción es:



Codigo en Java:

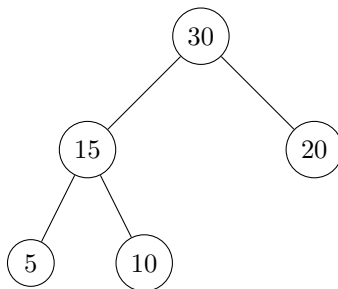
```
1 public class Main {
2     public static void main(String[] args) {
3         MinHeap<Integer> minHeap = new MinHeap<>(10);
4
5         minHeap.encolar(5);
6         minHeap.encolar(10);
7         minHeap.encolar(20);
8         minHeap.encolar(30);
9         minHeap.encolar(15);
10
11         System.out.println("Heap antes de insertar:");
12         minHeap.imprimirHeap();
13
14         minHeap.encolar(12);
15
16         System.out.println("Heap despues de insertar:");
17         minHeap.imprimirHeap();
18     }
19 }
```

Ejercicios Intermedios

Ejercicio 6: Construcción de un Max-Heap desde un Arreglo

Dado el arreglo {5, 10, 20, 30, 15}, construya un Max-Heap directamente utilizando el metodo de ajuste hacia abajo.

Solucion: El Max-Heap resultante es:



Codigo en Java:

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] arreglo = {5, 10, 20, 30, 15};
4         MaxHeap heap = new MaxHeap(arreglo);
5
6         System.out.println("Max-Heap construido desde el arreglo:");
7         heap.imprimirHeap();
8     }
9 }
10
11 class MaxHeap {
12     private int[] heap;
13     private int tamaño;
14
15     public MaxHeap(int[] arreglo) {
16         this.heap = arreglo;
17         this.tamaño = arreglo.length;
18         construirHeap();
19     }
20
21     private void construirHeap() {
22         for (int i = (tamaño / 2) - 1; i >= 0; i--) {
23             ajustarAbajo(i);
24         }
25     }
26
27     private void ajustarAbajo(int i) {
28         int mayor = i;
29         int izq = 2 * i + 1;
30         int der = 2 * i + 2;
31
32         if (izq < tamaño && heap[izq] > heap[mayor]) {
33             mayor = izq;
34         }
35         if (der < tamaño && heap[der] > heap[mayor]) {
36             mayor = der;
37         }
38         if (mayor != i) {
39             intercambiar(i, mayor);
40             ajustarAbajo(mayor);
41         }
42     }
43
44     private void intercambiar(int i, int j) {
45         int temp = heap[i];
46         heap[i] = heap[j];
47         heap[j] = temp;
48     }
49
50     public void imprimirHeap() {
51         for (int i = 0; i < tamaño; i++) {
52             System.out.print(heap[i] + " ");
53         }
54         System.out.println();
55     }
56 }

```

Ejercicio 7: Ordenamiento con HeapSort

Dado el arreglo {10, 20, 5, 30, 15}, ordene los elementos utilizando el algoritmo HeapSort.

Solucion: El arreglo ordenado es: {5, 10, 15, 20, 30}.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         int[] arreglo = {10, 20, 5, 30, 15};
4         HeapSort.heapSort(arreglo);
5
6         System.out.println("Arreglo ordenado con HeapSort:");
7         for (int num : arreglo) {
8             System.out.print(num + " ");
9         }
10    }
11 }
12
13 class HeapSort {
14     public static void heapSort(int[] arreglo) {
15         int n = arreglo.length;
16
17         for (int i = n / 2 - 1; i >= 0; i--) {
18             ajustarAbajo(arreglo, n, i);
19         }
20
21         for (int i = n - 1; i > 0; i--) {
22             intercambiar(arreglo, 0, i);
23             ajustarAbajo(arreglo, i, 0);
24         }
25     }
26
27     private static void ajustarAbajo(int[] arreglo, int n, int i) {
28         int mayor = i;
29         int izq = 2 * i + 1;
30         int der = 2 * i + 2;
31
32         if (izq < n && arreglo[izq] > arreglo[mayor]) {
33             mayor = izq;
34         }
35         if (der < n && arreglo[der] > arreglo[mayor]) {
36             mayor = der;
37         }
38         if (mayor != i) {
39             intercambiar(arreglo, i, mayor);
40             ajustarAbajo(arreglo, n, mayor);
41         }
42     }
43
44     private static void intercambiar(int[] arreglo, int i, int j) {
45         int temp = arreglo[i];
46         arreglo[i] = arreglo[j];
47         arreglo[j] = temp;
48     }
49 }
```

Ejercicio 8: Fusion de Dos Heaps

Dado dos Max-Heaps representados por {40, 20, 10} y {30, 15, 5}, fusi6nelos en un solo Max-

Heap.

Solucion: El Max-Heap resultante es {40, 30, 20, 15, 10, 5}.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         int[] heap1 = {40, 20, 10};
4         int[] heap2 = {30, 15, 5};
5
6         MaxHeapFusion fusion = new MaxHeapFusion(heap1, heap2);
7         fusion.imprimirHeap();
8     }
9 }
10
11 class MaxHeapFusion {
12     private int[] heap;
13     private int tamaño;
14
15     public MaxHeapFusion(int[] heap1, int[] heap2) {
16         this.tamaño = heap1.length + heap2.length;
17         this.heap = new int[tamaño];
18
19         System.arraycopy(heap1, 0, heap, 0, heap1.length);
20         System.arraycopy(heap2, 0, heap, heap1.length, heap2.length);
21         construirHeap();
22     }
23
24     private void construirHeap() {
25         for (int i = (tamaño / 2) - 1; i >= 0; i--) {
26             ajustarAbajo(i);
27         }
28     }
29
30     private void ajustarAbajo(int i) {
31         int mayor = i;
32         int izq = 2 * i + 1;
33         int der = 2 * i + 2;
34
35         if (izq < tamaño && heap[izq] > heap[mayor]) {
36             mayor = izq;
37         }
38         if (der < tamaño && heap[der] > heap[mayor]) {
39             mayor = der;
40         }
41         if (mayor != i) {
42             intercambiar(i, mayor);
43             ajustarAbajo(mayor);
44         }
45     }
46
47     private void intercambiar(int i, int j) {
48         int temp = heap[i];
49         heap[i] = heap[j];
50         heap[j] = temp;
51     }
52
53     public void imprimirHeap() {
54         for (int i = 0; i < tamaño; i++) {
```

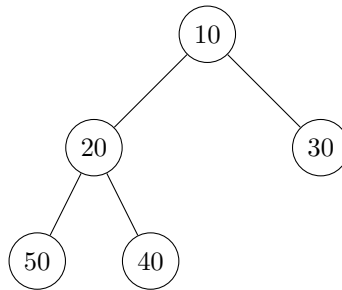
```
55         System.out.print(heap[i] + " ");
56     }
57     System.out.println();
58 }
59 }
```

Ejercicios Dificiles

Ejercicio 11: Construcción de un Min-Heap desde una Lista Dinámica

Dada la lista {50, 40, 30, 20, 10}, construya un Min-Heap de forma dinámica mientras inserta los elementos uno por uno.

Solución: El Min-Heap resultante es:



Código en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         MinHeap<Integer> heap = new MinHeap<>(10);
4
5         System.out.println("Insertando elementos:");
6         heap.encolar(50);
7         heap.encolar(40);
8         heap.encolar(30);
9         heap.encolar(20);
10        heap.encolar(10);
11
12        System.out.println("Min-Heap resultante:");
13        heap.imprimirHeap();
14    }
15 }
```

Ejercicio 12: Implementación de una Cola de Prioridad con Heaps

Implemente una cola de prioridad utilizando un Max-Heap y realice las siguientes operaciones:

1. Insertar {10, 40, 30, 50, 20}.
2. Extraer los dos elementos con mayor prioridad.

Solución:

1. Después de insertar los elementos, el Max-Heap es {50, 40, 30, 10, 20}.

2. Los elementos extraídos son 50 y 40.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         MaxHeap<Integer> heap = new MaxHeap<>(10);
4
5         heap.encolar(10);
6         heap.encolar(40);
7         heap.encolar(30);
8         heap.encolar(50);
9         heap.encolar(20);
10
11        System.out.println("Heap despues de insertar:");
12        heap.imprimirHeap();
13
14        System.out.println("Elemento con mayor prioridad: " + heap.desencolar());
15        System.out.println("Elemento con segunda mayor prioridad: " + heap.
16            desencolar());
17
18        System.out.println("Heap despues de extracciones:");
19        heap.imprimirHeap();
20    }
21 }
```

Ejercicio 13: Reconstruccion de un Heap despues de una Eliminacion Arbitraria

Dado el Max-Heap {50, 30, 40, 10, 20}, elimine el elemento 30 (no la raíz) y reconstruya el heap.

Solucion: El Max-Heap resultante es {50, 20, 40, 10}.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         MaxHeap<Integer> heap = new MaxHeap<>(10);
4
5         heap.encolar(50);
6         heap.encolar(30);
7         heap.encolar(40);
8         heap.encolar(10);
9         heap.encolar(20);
10
11        System.out.println("Heap antes de eliminacion:");
12        heap.imprimirHeap();
13
14        heap.eliminar(30);
15
16        System.out.println("Heap despues de eliminar 30:");
17        heap.imprimirHeap();
18    }
19 }
```

Ejercicio 14: Creacion de un Heap K-Ario

Implemente un Heap K-Ario (donde cada nodo tiene K hijos) para K=3 y realice las siguientes operaciones:

1. Insertar {10, 20, 30, 40, 50, 60, 70, 80}.

2. Extraer la raiz.

Solucion: El Heap K-Ario despues de las operaciones es {70, 60, 50, 40, 20, 10, 30}.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         KHeap<Integer> heap = new KHeap<>(3);
4
5         heap.encolar(10);
6         heap.encolar(20);
7         heap.encolar(30);
8         heap.encolar(40);
9         heap.encolar(50);
10        heap.encolar(60);
11        heap.encolar(70);
12        heap.encolar(80);
13
14        System.out.println("Heap K-Ario antes de extraccion:");
15        heap.imprimirHeap();
16
17        System.out.println("Elemento extraido: " + heap.desencolar());
18
19        System.out.println("Heap K-Ario despues de extraccion:");
20        heap.imprimirHeap();
21    }
22 }
```

Ejercicio 15: Simulacion de un Algoritmo de Heap para Multiples Colas

Simule la gestion de 3 colas de prioridad utilizando un Max-Heap para almacenar tareas con sus tiempos de ejecucion. Inserte las tareas {T1: 50, T2: 30, T3: 40, T4: 20, T5: 10}, y procese las dos tareas con mayor prioridad.

Solucion: Las tareas procesadas son T1 y T3. El Max-Heap resultante es {30, 20, 10}.

Codigo en Java:

```
1 public class Main {
2     public static void main(String[] args) {
3         MaxHeap<Tarea> heap = new MaxHeap<>(10);
4
5         heap.encolar(new Tarea("T1", 50));
6         heap.encolar(new Tarea("T2", 30));
7         heap.encolar(new Tarea("T3", 40));
8         heap.encolar(new Tarea("T4", 20));
9         heap.encolar(new Tarea("T5", 10));
10
11        System.out.println("Tareas procesadas:");
12        System.out.println(heap.desencolar());
13        System.out.println(heap.desencolar());
14
15        System.out.println("Heap despues de procesar tareas:");
16        heap.imprimirHeap();
17    }
18 }
19
20 class Tarea implements Comparable<Tarea> {
21     String nombre;
22     int prioridad;
```

```

23
24     public Tarea(String nombre, int prioridad) {
25         this.nombre = nombre;
26         this.prioridad = prioridad;
27     }
28
29     @Override
30     public int compareTo(Tarea otra) {
31         return Integer.compare(this.prioridad, otra.prioridad);
32     }
33
34     @Override
35     public String toString() {
36         return nombre + " (" + prioridad + ")";
37     }
38 }

```

Bonus: Ordenamientos con Heaps

En esta seccion se presentan diferentes metodos para realizar ordenamientos utilizando Heaps. Cada metodo toma una lista de elementos y retorna otra lista con los elementos ordenados.

Metodo 1: HeapSort con Max-Heap

Dado un Max-Heap, implemente un metodo para ordenar una lista en orden ascendente.

Solucion:

1. Construya un Max-Heap con los elementos de la lista.
2. Extraiga los elementos del Max-Heap uno por uno y coloquelos en la lista de salida.

Codigo en Java:

```

1 public class HeapSort {
2     // Metodo principal para ordenar una lista en orden ascendente utilizando Max-
      Heap
3     public static void heapSortAscendente(int[] arreglo) {
4         int n = arreglo.length;
5
6         // Construir el Max-Heap
7         for (int i = n / 2 - 1; i >= 0; i--) {
8             ajustarAbajo(arreglo, n, i); // Ajustar hacia abajo desde el indice
              actual
9         }
10
11        // Extraer elementos uno por uno
12        for (int i = n - 1; i > 0; i--) {
13            intercambiar(arreglo, 0, i); // Mover la raiz al final del arreglo
14            ajustarAbajo(arreglo, i, 0); // Ajustar el Max-Heap para el nuevo tamano
15        }
16    }
17
18    // Ajusta un nodo hacia abajo para mantener la propiedad del Max-Heap
19    private static void ajustarAbajo(int[] arreglo, int n, int i) {

```

```

20     int mayor = i; // Inicializar el mayor como la raiz
21     int izq = 2 * i + 1; // Hijo izquierdo
22     int der = 2 * i + 2; // Hijo derecho
23
24     // Comparar con el hijo izquierdo
25     if (izq < n && arreglo[izq] > arreglo[mayor]) {
26         mayor = izq;
27     }
28     // Comparar con el hijo derecho
29     if (der < n && arreglo[der] > arreglo[mayor]) {
30         mayor = der;
31     }
32     // Intercambiar y continuar ajustando si el mayor no es la raiz
33     if (mayor != i) {
34         intercambiar(arreglo, i, mayor);
35         ajustarAbajo(arreglo, n, mayor);
36     }
37 }
38
39 // Intercambia dos elementos en el arreglo
40 private static void intercambiar(int[] arreglo, int i, int j) {
41     int temp = arreglo[i];
42     arreglo[i] = arreglo[j];
43     arreglo[j] = temp;
44 }
45
46 public static void main(String[] args) {
47     int[] arreglo = {10, 20, 15, 30, 40};
48     heapSortAscendente(arreglo);
49     System.out.println("Lista ordenada en orden ascendente:");
50     for (int num : arreglo) {
51         System.out.print(num + " ");
52     }
53 }
54 }

```

Metodo 2: HeapSort con Min-Heap para Orden Descendente

Dado un Min-Heap, implemente un metodo para ordenar una lista en orden descendente.

Solucion:

1. Construya un Min-Heap con los elementos de la lista.
2. Extraiga los elementos del Min-Heap uno por uno y coloquelos en la lista de salida.

Codigo en Java:

```

1 public class HeapSort {
2     // Metodo principal para ordenar una lista en orden descendente utilizando Min-Heap
3     public static void heapSortDescendente(int[] arreglo) {
4         MinHeap heap = new MinHeap(arreglo.length);
5
6         // Insertar todos los elementos en el Min-Heap
7         for (int num : arreglo) {
8             heap.encolar(num);

```

```

9         }
10
11         // Extraer los elementos del Min-Heap y colocarlos en el arreglo
12         for (int i = 0; i < arreglo.length; i++) {
13             arreglo[i] = heap.desencolar();
14         }
15     }
16
17     public static void main(String[] args) {
18         int[] arreglo = {10, 20, 15, 30, 40};
19         heapSortDescendente(arreglo);
20         System.out.println("Lista ordenada en orden descendente:");
21         for (int num : arreglo) {
22             System.out.print(num + " ");
23         }
24     }
25 }

```

Explicacion Detallada de las Funciones de Heap

Funcion encolar

La funcion `encolar` inserta un nuevo elemento al final del heap y lo ajusta hacia arriba para mantener la propiedad del heap.

Codigo en Java:

```

1 public void encolar(int elemento) {
2     if (tamanio >= heap.length) {
3         throw new IllegalStateException("Heap lleno"); // Validar espacio disponible
4     }
5     heap[tamanio] = elemento; // Insertar el elemento al final
6     subir(tamanio); // Ajustar hacia arriba
7     tamanio++; // Incrementar el tamanio del heap
8 }
9
10 // Metodo para ajustar un nodo hacia arriba
11 private void subir(int indice) {
12     while (indice > 0) {
13         int padre = (indice - 1) / 2; // Calcular el indice del nodo padre
14         if (heap[indice] <= heap[padre]) { // Si el nodo es menor o igual al padre,
15             detener
16             break;
17         }
18         intercambiar(indice, padre); // Intercambiar con el padre
19         indice = padre; // Actualizar el indice al del padre
20     }
21 }

```

Funcion desencolar

La funcion `desencolar` elimina la raiz del heap (el mayor o menor elemento) y ajusta el heap hacia abajo para restablecer la propiedad del heap.

Codigo en Java:

```

1 public int desencolar() {
2     if (tamanio == 0) {
3         throw new IllegalStateException("Heap vacio"); // Validar que el heap no
4             este vacio
5     }
6     int raiz = heap[0]; // Guardar el elemento raiz
7     heap[0] = heap[--tamanio]; // Reemplazar la raiz con el ultimo elemento
8     ajustarAbajo(0); // Ajustar el heap hacia abajo
9     return raiz; // Retornar la raiz extraida
10 }
11 // Metodo para ajustar un nodo hacia abajo
12 private void ajustarAbajo(int indice) {
13     while (indice < tamanio / 2) {
14         int izquierdo = 2 * indice + 1; // Hijo izquierdo
15         int derecho = 2 * indice + 2; // Hijo derecho
16         int mayor = izquierdo; // Inicializar con el hijo izquierdo
17
18         // Comparar con el hijo derecho
19         if (derecho < tamanio && heap[derecho] > heap[izquierdo]) {
20             mayor = derecho;
21         }
22         // Si el nodo actual es mayor o igual al mayor hijo, detener
23         if (heap[indice] >= heap[mayor]) {
24             break;
25         }
26         intercambiar(indice, mayor); // Intercambiar con el mayor hijo
27         indice = mayor; // Actualizar el indice al del mayor hijo
28     }
29 }

```

Funcion ajustar

La funcion ajustar restablece la propiedad del heap desde un nodo especifico hacia abajo.

Codigo en Java:

```

1 private void ajustar(int indice) {
2     while (indice < tamanio / 2) {
3         int izquierdo = 2 * indice + 1; // Hijo izquierdo
4         int derecho = 2 * indice + 2; // Hijo derecho
5         int mayor = izquierdo; // Inicializar con el hijo izquierdo
6
7         // Comparar con el hijo derecho
8         if (derecho < tamanio && heap[derecho] > heap[izquierdo]) {
9             mayor = derecho;
10        }
11        // Si el nodo actual es mayor o igual al mayor hijo, detener
12        if (heap[indice] >= heap[mayor]) {
13            break;
14        }
15        intercambiar(indice, mayor); // Intercambiar con el mayor hijo
16        indice = mayor; // Actualizar el indice al del mayor hijo
17    }
18 }

```